A Single Dummy Machine

Bot architecture and research league development plan

Version 4 | 8 November 2026 | Max Schireson

1 Purpose and direction

The aim is to build a bridge-playing system that chooses coherent strategies under uncertainty. Given its hand, dummy, the auction and the play so far, it should consider plausible opposing layouts, compare plans that can actually be followed without seeing those layouts, and adapt when play supplies new information. Ultimately both declarer and defenders should reason this way.

Build a hybrid player that combines learned beliefs and fast policies with selective information-consistent strategy search and an efficient double-dummy backend. Develop it inside a persistent research league: self-play improves networks, engineering agents test algorithmic changes, and an independent arena controls promotion. Preserve old bots as benchmarks, teachers, partners and specialists.

This revision connects the likely strong-bot architecture to a staged development path. It incorporates the work in progress on PIMC to consistent strategies using alpha mu and on solver information reuse across related layouts. These workstreams are research inputs, not established general strength or speedup claims. Population size, model capacity and search budgets remain measured choices.

What the first experiments should establish

- Can a small strategy solver correctly value discovery, timing and exit plays that ordinary perfect-information Monte Carlo handles poorly?
- Can one proof or strategy cover a substantial set of related deals, after including the cost of finding, checking and refining that generalization?
- Can a fast policy learn from stronger search and league experience while keeping expensive reasoning selective and preserving performance against historical opponents?
- Does modelling uncertainty about opponent ability and style improve both card inference and practical play?

Scope and current resources

Start with card play in supplied contracts and auctions. Build on the working harness and PIMC baseline while validating small strategy problems, then improve full-hand declarer play and defensive coordination. Keep bidding fixed while measuring card-play changes; bidding research, a polished interface and tournament deployment remain later extensions.

The current hardware plan is a 128 GB MacBook Pro followed by one 512 GB M5 Ultra Mac Studio. Frontier models assist with engineering and review; local models can take on more work as their usefulness is demonstrated. Additional machines remain optional. Sections 10 and 11 give explicit compute and data scenarios rather than throughput promises.

2 The decision problem

A layout is a complete assignment of the cards. An information set contains the histories a player cannot distinguish using the information legally available to that player. A belief assigns probabilities to possible layouts and, where relevant, to opponent types. A strategy maps the player's available information and remembered observations to an action or action distribution.

The central requirement is simple: **indistinguishable histories must produce the same action distribution**. A strategy may finesse differently after a defender shows out; it may not finesse differently merely because the simulator knows where the queen is. Randomized play is allowed, but its random choices must not reveal concealed cards or future random outcomes.

What is being optimized

Expected tricks is a useful initial objective. Contract-making probability and expected IMPs answer different questions and should be selectable, explicit objectives. An IMP objective needs a reference result or reference-score distribution on the same board; a matchpoint objective needs an explicit field model. Optimizing one and reporting another is not sufficient.

For a fixed strategy, the value against a specified opponent model is the belief-weighted expectation of the resulting game score. The expectation includes the model's action probabilities and any randomized choices in our own policy. Candidate ranking should use this executable strategy value, including its fallback and replanning behaviour.

Quantity	Meaning and limitation
Double-dummy value	The perfect-information result for one layout. Useful for search, labels and bounds in the appropriate game; it is not a single-dummy decision value.
Value against omniscient best defence	The value of our fixed information-consistent strategy against a true best response that sees the layout. Exact computation gives a pessimistic bound against information-limited defenders.
Value against an opponent model	Expected performance against specified information-limited policies. This is a model estimate, not a bound on real opponents.
Best strategy found	The highest-valued candidate searched so far. It need not be the globally optimal strategy.

Three distinct correctness claims

Information consistency means the policy uses no concealed information. Exact evaluation means the value of that policy is computed without approximation under stated assumptions. Optimality means no admissible policy can do better. These claims should be tested and reported separately.

Enumerating every remaining layout removes layout-sampling error. It does not by itself remove continuation-search error, opponent-model error or the limitations of a template library. An exact small-ending reference therefore needs a precise game definition and independent strategy enumeration or another justified exact algorithm.

3 Illustrative play capabilities

The examples below suggest useful tests and candidate generators. They are a subset of bridge play, not an exhaustive taxonomy or a requirement that the engine classify every move. One card may serve several purposes, and an unfamiliar successful strategy should remain admissible.

Capability	Examples	What the planner must retain
Suit decisions	Finesse versus drop; two-way finesse; restricted choice	Joint layout probabilities, available entries and the chosen scoring objective.
Discovery	Count a side suit; delay a guess; learn from a return	How observations change later choices, together with the cost of obtaining them.
Timing and entries	Unblock; preserve a re-entry; choose which hand wins	Future reachability of useful plays, not just immediate trick winners.
Exits and endplays	Strip and throw in; avoidance; repeated exits	Whether a concession is forced, requires a guess, or relies on a modelled error.
Safety and alternatives	Protect the contract; combine chances in two suits	The full score distribution and which chances survive each commitment.
Squeezes and pressure	Simple squeeze; strip squeeze; later more complex endings	Guard dependencies, timing, entries and the information revealed by discards.
Deception	Falsecard; conceal length; encourage a poor return	The defender's interpretation and subsequent response under an explicit model.
Defensive coordination	Count or attitude signals; unblock for partner; choose a return	Different private hands, shared agreements and information also visible to declarer.

A small example of a contingent strategy

Suppose declarer must eventually choose a direction for a queen finesse. Cashing a side suit first may reveal a void or enough length information to change the odds. A candidate strategy could preserve the required entry, play the side suit, update the distribution of the queen, and then choose the finesse direction from the observed history.

That strategy should be compared with guessing immediately and with any alternative that avoids the guess. Its value depends on the actual holding: cashing the suit may consume an entry, permit a ruff, give away the lead or disclose information to the defenders. No generic probability gain is assumed.

A useful test family holds declarer and dummy fixed while varying the defensive cards. Some layouts yield the same observations and must retain the same decision. Other layouts separate after play. The test should include cases where discovery helps, cases where it costs too much, and cases where a different strategy dominates both choices.

4 Target architecture of a strong bot

The likely end state is a hybrid player. Compact learned components handle familiar decisions, maintain beliefs and guide a selective planner. The planner evaluates executable strategies across possible layouts, using batched exact solving and reusable information where sound. A larger offline teacher generates better targets than the deployed player can afford. This is a working architectural hypothesis to test, not a claim that every component is necessary.

Layer	Role in a strong bot	Starting point
Information boundary	One legal observation and remembered history per acting player; explicit objective and agreements.	Existing game engine, legal moves and replay.
Belief and player inference	Constrained joint layouts with uncertainty over opponent style, capability and partner conventions.	Cards and voids, then auction and action likelihoods.
Policy and value	Fast legal action prior, fallback policy and information-state continuation estimates.	Small models distilled from search and play.
Strategy planner	Shared choices at indistinguishable histories; observation branches and explicit continuation rules.	Alpha-mu-inspired declarer search and exact tiny cases.
Solver backend	Batch related layouts; reuse safe bounds, proofs and ordering information with validity checks.	Pinned DDS and a fresh-solve baseline.
Compute controller	Choose cheap play, more samples, deeper search or an ending solver from decision uncertainty.	Simple measured thresholds before a learned controller.
Persistent plan state	Retain contingencies and caches, update after observations and replan when needed.	Replayable plan with a specified fallback.

Interfaces that keep the pieces replaceable

The actor receives a legal observation, role, contract objective and known agreements. Belief updates return weighted valid layouts and type uncertainty. Search returns an action, an executable continuation and diagnostics describing approximations and budget. The solver accepts hypothetical positions and returns exact results or explicitly qualified bounds. Record model, policy and cache versions at every boundary.

These are logical interfaces, not a requirement for separate services. Start in one process with a compact registry and job store. Network training, coding agents and experiment scheduling belong to the development system; the playing bot need not invoke a language model for each card.

4 Decision making and deployment

A decision from observation to action

On each public action, update the acting player's own information and joint belief. A cheap policy proposes legal actions and supplies a continuation. If the move is forced, or a tested confidence rule says further computation rarely matters, act cheaply. Otherwise compare candidate strategies, expanding observations that could change a later choice. Allocate more effort when the score stakes, uncertainty or disagreement justify it.

Issue related hypothetical layouts to the solver together. Reuse results only under their stated validity conditions. As observations separate worlds, prune or reweight them and preserve distinctions that still affect strategy. At budget expiry, execute the best evaluated admissible continuation or the declared fallback. Evaluate exactly this replanning and timeout behavior.

Near the end, use more complete information-set search when affordable. A cheap double-dummy ending is not automatically a cheap single-dummy ending. No switch to exact single-dummy reasoning is triggered solely by reaching a fixed number of remaining cards.

Three budgets for the same core player

Mode	Purpose	Budget and evidence
Interactive player	Strong decisions with predictable response time.	Selective search, latency limits and a reliable fallback.
Self-play actor	Generate useful diverse experience at high aggregate throughput.	Mostly cheap play; exploration and selective teacher queries.
Offline teacher	Improve labels, investigate failures and test the value of deeper reasoning.	Larger samples and search budgets; retain executable strategy targets.

Robustness and the two defenders

A strong player should perform well against a broad reference mixture, then exploit opponent-specific evidence when it is reliable. Compare adaptive exploitation with a fixed robust policy and retain broad belief support when an opponent is unfamiliar. Success against the league is evidence about that population, not a certificate of low exploitability against all legal strategies.

Declarer controls both visible hands after dummy appears. Each defender instead acts from its own private hand and public observations. Shared weights and shared agreements do not permit shared concealed cards. Defensive search uses separately conditioned response models; unrestricted recursive searches of every player's beliefs are not required. Limited-depth responses and learned continuations are the initial approximation.

The simulator owns the true deal. Hidden cards may label belief training or inform a separate training critic, but cannot reach the deployed actor, its cache keys or its stopping rule as privileged truth. Plans and macros must expand into legal choices under this same boundary.

5 Searching over consistent strategies

The search compares executable contingent strategies across possible layouts. It can combine ordinary card expansion, optional bridge patterns and proposals suggested by DD analysis. Keep a card-level route: useful discovery plays need not occur in the selected perfect-information principal variations, and a pattern library will be incomplete.

PIMC to consistent strategies work in progress

An active workstream is adapting the PIMC engine toward common decisions on indistinguishable histories, drawing on alpha mu by Tristan Cazenave and Veronique Ventos and its optimizations with Swann Legras. The published approach maintains achievable outcome vectors and Pareto fronts; its opponent model gives defenders perfect information. It is a relevant starting point for addressing strategy fusion and non-locality, not a complete model of information-limited bridge defense. [2,3]

The current direction is a fresh implementation informed by those papers, with explicit attribution. Project discussions report initial tiny-ending correctness and replay checks; broader playing-strength evidence remains preliminary. Keep this work marked as a prototype until a pinned implementation, reproducible tests and held-out comparisons establish the claims needed for promotion. Do not assume alpha-mu source code is Apache-licensed merely because DDS is.

Candidate generation and evaluation

Develop legal alternatives on shared information sets and retain observation-conditioned branches. Against a fixed stochastic defender model, average over weighted layouts and that model's responses. Against omniscient best defense, search the worst legal response to our fixed strategy. DDS alone does not compute that value because ordinary DD search also gives declarer the hidden layout.

The initial alpha-mu comparison should use the same beliefs, scoring target and elapsed budget as PIMC, plus a larger-budget teacher. Validate tiny cases independently and inspect executable continuations. If search stops with DD tail values, report the residual perfect-information approximation; a common root action does not remove fusion throughout the rest of the hand.

Budget allocation and continuity

Allocate more worlds, deeper continuations or stronger response evaluation when they can change the preferred strategy. Every intervening opponent action matters unless forced or represented by a justified transition. Discards during a macro can reveal information, consume entries or alter the later guess.

A strategy includes its fallback, replanning triggers and behavior after unanticipated observations. Evaluate those same rules. Exact layout enumeration removes sampling error but not search truncation, model error or a restricted strategy set. Record these approximations separately so later learning improves the actual bottleneck.

6 Beliefs and player information

Belief construction should start with constraints that are genuinely certain: known cards, cards already played, remaining hand sizes and demonstrated voids. Auction and play tendencies usually supply likelihoods, with uncertainty about the bidding system, the player and the reliability of the model.

The perspectives cannot share the same concealed cards

If South declares after dummy appears, South's outer sample fixes North and South and varies East-West. West's internal belief instead fixes West and North and varies South-East. Filtering South's outer sample is insufficient for West, because every layout in that sample already fixes South's concealed hand. Before the opening lead, dummy is not yet visible and the defender's view is different again.

This distinction should be covered by explicit invariance tests: changing information unavailable to a player must not change that player's action distribution when the visible history stays the same. The tests should cover learned policies, opponent models, macros, cached responses and replanning.

Sampling and updating

A constructive sampler can satisfy card and shape constraints, then weight proposals by an auction and play model. Learned proposals can improve efficiency later. If the proposal distribution changes, importance weights must include the target-to-proposal ratio; observation likelihood alone is not generally the right weight. If tractable proposal probabilities are unavailable, use a sampling method whose correction is justified.

The posterior should preserve dependencies between important unknowns. Card marginals alone can miss a connection between shape, an honour location and an entry. Explicit joint variables may suffice for some plans; weighted full layouts provide a fallback when the abstraction is too coarse.

Effective sample size is useful but cannot detect every missing mode. Also inspect whether strategically different possibilities have support, whether answers stabilize with fresh samples, and whether rare alternatives change the preferred strategy. Adaptive sample counts should depend on the decision, not merely the number of cards left.

Opponent actions are evidence

The likelihood of a defender's action is evaluated from that defender's information and assumed policy. A lead, return or discard is informative even without an explicit signalling agreement. Intentional signalling and falsecarding add further structure; neither should be treated as perfectly truthful or completely uninformative by default.

Use a small deviation model to avoid ruling out plausible layouts after an unusual legal action. This does not repair a wrong hard constraint. Calibrate on the intended population and important play situations. Choose either an incremental likelihood update or an amortized posterior over the whole history; applying the same action likelihood again to a posterior that already includes it double-counts evidence.

7 Opponent capability as a latent variable

Opponent capability is part of the uncertainty. A surprising discard might reflect an unusual holding, a missed tactic, an agreement or deception. Assuming one skill level can therefore distort the inferred cards.

Start with a modest population

Try roughly 8 to 16 type descriptors initially, combining capability bands with selected behavioral differences. These latent types are distinct from the number of archived bots. Vary search budgets, tactical patterns and policy families; random mistakes added to one strong policy will miss systematic weaknesses. Historical and specialist league members supply useful diversity.

Dimension	Illustrative variation
Tactical capability	Routine play; competent combination play; stronger search or learned policies.
Information use	How reliably the player counts, draws inferences and adapts to new evidence.
Agreements	Bidding system, opening-lead conventions and partnership carding.
Style and reliability	Active versus passive defence, signal reliability, falsecard tendencies and error rate.

Avoid taking the full Cartesian product of these dimensions initially. Keep conventions separate from strength, allow the two defenders to have different capabilities, and give partnership agreements a shared representation. Omniscient defence belongs in stress testing and bound computation, not in the ordinary population of legal opponents.

Infer a mixture rather than immediately choosing one type

Maintain a joint posterior over layout x and opponent type t: posterior(x,t) is proportional to prior(x,t) times the likelihood of the observed auction and play under that layout and type. The same type persists across a hand; it should not be independently redrawn for each observed card. Across repeated encounters, evidence can update a session-level prior.

A practical first model is a shared action-likelihood network conditioned on a type descriptor, plus a shared belief proposal network with the same conditioning. Evaluate or sample a few plausible types and combine their layout distributions using posterior type weights. Preserve type-layout correlations through joint particles or separate conditional components. An averaged type embedding alone may lose them.

Hard assignment to the most likely type can be a later speed optimization when confidence is high. Keep some broad fallback support for opponents unlike the training population. Known public agreements can be supplied directly; a hidden simulator type label must not be exposed to the actor as if observed.

Start with fixed information-limited responses. Shared models, sampled joint type pairs and cached responses avoid recursively nesting a full planner at every decision. Later, train both defenders with separate observations and shared agreements, and test limited-depth responses. Self-play alone does not certify an equilibrium or solve partnership coordination.

How to judge whether it helps

Compare a generic opponent, a fixed mixture, an inferred mixture and an oracle with the true simulated type. Measure prediction, belief calibration and play against held-out policies and styles. The oracle gap indicates the potential value of better type inference. Bayesian opponent modelling provides relevant precedent. [8]

8 Solver reuse across related deals

The priority workload fixes declarer and dummy and varies the defensive allocation. The work in progress extends the solver interface and investigates which information from one solve can accelerate others. Related-world batches are also a natural workload inside consistent-strategy search, so the planner and solver should share a stable batch interface while remaining independently testable.

Validate the entire solve set first

A batch specifies the common fixed hands, alternate opposing holdings, contract or strain, player to act, current trick and any played-card context. Before solving, check card encoding, duplicates within each layout, correct remaining hand sizes, disjoint fixed and variable cards, common card universe, consistency of public history and equal fixed hands. Repeated cards across alternate worlds are expected; duplicated cards within one world are invalid. Define errors and output ordering before optimizing.

Reuse has several different meanings

Reuse class	Validity requirement	First measurement
Exact results and bounds	A complete compatible state key, solver semantics and correct bound direction.	Fresh-solve agreement, hit rate and net time.
Move ordering and hints	Hints may alter order but must not suppress required alternatives.	Nodes and elapsed time, including failed hints.
Generalized DD certificates	Explicit rank, ownership, shape and entry conditions sufficient for every reused conclusion.	Coverage, verification cost and adversarial counterexamples.
Common strategy or response	Same relevant information state, policy version and assumptions.	Legal execution and consistent choices across worlds.

A coarse similarity key may retrieve a hint or a certificate to check. It must not serve as a result-cache key without a sound equivalence argument. Identical DD values do not imply a common single-dummy strategy. Rank equivalences can also fail in information-sensitive play when spot cards convey signals.

Abstract only the proof conditions that are sufficient

Candidate conditions include West being void in diamonds or both defenders having at least two spades, rather than an unnecessarily exact shape. Preserve every condition needed for the conclusion and relevant opponent alternatives; facts absent from one principal variation are not automatically irrelevant. Ginsberg's partition search is a precedent. Counterexamples can refine a proposed certificate, but testing many instances alone does not prove universal validity. [1]

Measure useful reuse rather than successful probes

Compare fresh solves, safe retained-context baselines, ordering reuse and generalized proofs over many fixed-hand families. Include batches of 1, 8, 32, 128 and 1,000, unrelated-deal controls, auction constraints and later positions. Measure total time, nodes, memory, verification overhead and tails. Select or bypass reuse adaptively when its overhead loses on small batches. Generalized proof reuse remains experimental; no broad net speedup is assumed.

9 Learning roles and initial model sizes

Start small, and size components according to how often they run and what they must learn. The playing models need structured card and history representations; they do not need to be general-purpose language models. A fast actor may remain small even if a stronger teacher, belief model or proposal model benefits from more capacity.

Component	Initial parameter range	When it runs and what to test
Fast play policy	1 to 5 million	Every non-forced action. Compare a compact MLP or recurrent model with a small card/history transformer.
Belief proposal	5 to 20 million	On material belief updates, with incremental work between them. Check joint-layout coverage and calibration.
Opponent likelihood	2 to 10 million shared	Across candidate layouts and types. Share parameters and type embeddings; batch evaluations.
Training value or critic	5 to 30 million	During training and selected search evaluations. It can be larger than the deployed actor.
Strategy proposer	5 to 20 million	At replanning points. Add only after a useful training set of strategies exists.
DD approximator	5 to 30 million if needed	Where profiling justifies replacing some exact calls. Judge action regret and downstream effects, not only average error.

These are proposed starting ranges, not requirements to train six separate networks. A shared encoder with a few heads may reduce repeated work. Separate models make experiments easier to interpret and can run at different frequencies. Begin with the minimum set needed for the current stage.

How learning enters

Start with belief supervision on generated histories and imitation of useful decisions from exact tiny solvers or stronger strategy search. Then compare self-play policy improvement, search-generated targets and their combination. Train on the actual game objective. DD values are useful auxiliary labels; DD trick loss as the primary reward can discourage information plays and practical deception.

A training-time critic may see the full layout as well as the actor's history or belief state. Its target must remain the return of the information-limited policy. The actor must never receive the privileged layout or an unobserved opponent label. DouZero and PerfectDou are useful precedents for relatively compact game models and privileged training information, but their learning rates do not establish bridge requirements. [6,7]

When to make a component larger

Compare smaller and larger models on both equal data and equal wall-clock budgets. Escalate toward 20 to 100 million parameters only where capacity appears limiting. A larger model that gains per decision but reduces fresh experience substantially may slow learning. Conversely, a belief model that prevents expensive bad searches may justify its cost. The right sizes are unlikely to be uniform.

Include the learner in the budget. Forward plus backward work is often roughly three forward passes for dense layers. Reusing each new position k times therefore costs roughly 3k forward-equivalents, before optimizer overhead. This is a sizing approximation; measure actual training throughput and the benefit of additional replay.

9 Learning from league experience

Beliefs are the first learning target

For every generated position, the simulator supplies the real hidden holding as a training label while the network receives only the player's permitted observations. Self-play supplies histories that reveal how particular policies act with particular hands; the label does not depend on winning the board. Train a valid joint distribution, using an autoregressive or otherwise constrained sampler that retains shape and honor correlations.

Evaluate likelihood or other proper scoring rules, card and shape calibration, mode coverage and downstream play at equal elapsed time. Hold out complete deal families and entire generating-policy families. Better fit on familiar opponents is insufficient evidence of transferable card inference. Bridge belief-learning and Learned Belief Search provide useful precedents, while their published settings differ from this card-play task. [15,16]

Policy and value improvement

Distill the stronger planner's legal action distribution or selected executable strategy into the fast actor. Train information-state values from actual policy returns or justified strategy-search targets. Keep privileged training critics distinct from deployed value inputs. A DD outcome averaged over layouts is not automatically the value of an implementable continuation.

The feedback loop is search to better targets, targets to better networks, and networks to stronger or cheaper search. ReBeL is relevant for combining search and learning over beliefs, but its two-player zero-sum guarantees do not automatically extend to decentralized bridge defense. [13]

Control changes in the data generating population

Freeze actor snapshots and the partner/opponent mixture within an epoch. Tag replay with deal-family ID, role, legal observation, generating policy and model versions, agreements, search budget, sampling regime and hidden-card labels. Keep audit data out of training until explicitly retired. Learners can update continuously; published actors and arena contestants are immutable snapshots.

Mix recent experience with historical and strategically distinct play. Old histories remain useful for supervised belief training when their generating population is represented. Policy/value training additionally needs a stated treatment of stale trajectories, such as recent-policy filtering or a justified off-policy method. Track sampling and replay weights, and avoid counting repeated passes as fresh experience.

Learning partnership and computation

Test both dedicated defensive pairs and cross-play with independently trained or historical partners. Strong self-play conventions may fail with strangers. Fictitious Co-Play and Off-Belief Learning motivate diverse partner pools and explicit convention tests. [17,18]

Later, learn where extra search repays its cost, how to order solver moves, or which proposed proof is worth checking. These predictors may guide computation; they must not silently turn unproved outputs into exact bounds. Maintain a simple nonlearned fallback for each learned component.

10 Inference rates and the cost of a decision

Inference needs two measurements: low-batch latency for an interactive player and aggregate throughput for many simultaneous self-play games. A GPU can have good batched throughput and still be a poor choice for isolated tiny requests. CPU inference is a credible baseline for the smallest models.

Illustrative capacity arithmetic

For a rough dense-transformer calculation, take forward-pass work as about 2 times parameter count times processed tokens. With 64 tokens this gives the table below. The two compute columns assume an effective 1 or 5 TFLOP/s available to those operations. They are **sensitivity scenarios, not measured Mac inference rates**. Attention overhead, memory traffic, feature construction, sampling and framework dispatch can reduce realized throughput substantially.

Model	Approximate work	At 1 TFLOP/s	At 5 TFLOP/s
2 million parameters	0.26 GFLOP / evaluation	3,900 evaluations/s	19,500 evaluations/s
10 million parameters	1.28 GFLOP / evaluation	780 evaluations/s	3,900 evaluations/s
50 million parameters	6.4 GFLOP / evaluation	156 evaluations/s	780 evaluations/s

This supports exploring thousands of small-model evaluations per second under batching, with lower rates for larger models. It is not a forecast for a complete belief update or a searched card decision. An MLP over a compact feature vector has a different cost model; the same parameter count is not a universal inference unit.

Count all the work behind one card

An actor call may be cheap while the belief or opponent model dominates. For example, scoring 100 layouts under four types requires 400 likelihood evaluations if each is separate. A 10-million-parameter network at the illustrative 3,900 evaluations/s would then allow fewer than ten such updates per second before other work. Batching improves utilization; it does not remove this multiplication.

Possible remedies include caching per-information-set responses, incremental likelihood updates, sampling opponent types, using cheap explicit policies initially, or distilling a direct posterior proposal. A shared encoder may amortize public history while cheaper conditional heads handle layouts and types. Each remedy needs a coverage and decision-quality check.

An initial benchmark matrix

- Measure 1, 5, 20 and 50 million parameter models with the real inputs and outputs, including legal-action masks and history encoding.
- Test single requests and batches such as 32, 256 and 1,024; report latency percentiles, throughput and memory.
- Measure actor, belief, likelihood, sampling and solver work separately, then run them together with the learner.
- Repeat with a local coding model active. Use measured contention to decide scheduling rather than assuming CPU and GPU workloads are independent.

Search can run much less frequently than the fast actor. A slow high-quality teacher remains useful if its supervision improves enough subsequent decisions to repay its cost.

11 Self play throughput and learning volume

A complete bridge board contains 52 card actions; declarer controls 26 of them, including dummy. Forced moves may require no network call. For planning, let D be the measured aggregate card decisions per second across all workers, with the chosen policy, beliefs, opponents and search settings. Let u be the fraction of a day actually sustained at that rate after pauses, training and other work.

The approximate board rate is **86,400 times D times u divided by 52**. If D is already measured over the complete daily schedule, use u = 1. The following examples use u = 0.5 and assume all boards play to completion.

Aggregate decisions/s	Boards/day at half duty	Time for 1 million boards
10	8,300	About 120 days
100	83,000	About 12 days
1,000	830,000	About 1.2 days
10,000	8.3 million	About 3 hours

These are arithmetic scenarios, not platform benchmarks. Fast policy-only play may reach the higher rows; nested opponent search may fall near or below the lower ones. A 50 ms decision latency yields only 20 decisions/s if the machine completes one decision at a time. Parallel workers help only to the extent that compute and memory bandwidth remain available.

How much experience might be needed

Experiment	Proposed initial scale	What it can tell us
Teacher imitation	200,000 to 2 million selected positions	Whether a small policy can reproduce useful information-sensitive decisions.
First self-play learning curves	100,000 to 1 million fresh boards	Whether the training setup learns, remains calibrated and beats its initial policy.
Scaling study	1 to 10 million fresh boards	Whether gains persist and are limited by data, capacity, opponents or search quality.
Larger campaign	10 to 100 million boards only if justified	Whether continued compute has value; this is not an estimate of the amount required for expert play.

No reliable bridge-specific sample requirement is established here. Adjacent positions from one board, repeated layouts from one family and repeated passes through replay are correlated. Track fresh boards, distinct families, informative decisions and training updates separately. Learning curves should report quality against both new data and elapsed compute.

Use two experience streams: inexpensive broad league play and selective expensive teacher analysis of consequential or disputed positions. Sweep the teacher fraction and label budget. Track fresh families, useful decisions, validation cost and verified improvement per compute hour. Freeze the generating population within epochs, retain historical coverage and correct deliberate sampling bias where feasible.

12 Independent arena and evidence

The arena answers whether a candidate is correct, stronger, useful as a specialist or cheaper at comparable strength. It is independent of the code and weights under test. The scoring, deal generation, information boundary, stopping rules and promotion policy have their own versions. Candidate authors can propose evaluator changes, but cannot change the evaluator that judges their current candidate.

A staged evaluation

Stage	Evidence	Disposition
Correctness	Legal play, valid samples, replay, hidden-information invariance and solver agreement.	Reject invalid artifacts before expensive matches.
Development screen	Component metrics, targeted failures and small paired tournaments.	Terminate weak trials or fund confirmation.
Fresh confirmation	New concealed deal families and frozen reference opponents at matched budgets.	Estimate meaningful gains with uncertainty.
Promotion and audit	Historical anchors, unfamiliar policies, regression slices and an independent audit stream.	Promote, retain as a specialist, defer or archive.

Correctness claims remain separate

Use independent tiny-ending strategy enumeration or another justified exact reference. Test indistinguishable histories, discovery costs, misleading signals, entry timing and equal-DD-value cases requiring incompatible guesses. Reuse checks include adversarial layouts and fresh solves. Report information consistency, exact evaluation and optimality separately; a legal actor can still use an approximate search value.

Paired bridge comparisons

Compare declarer candidates on the same deals, supplied auctions, contracts and opposing policies. Compare defensive pairs separately against common declarers, and include mixed-partner tests. Use full duplicate comparisons for integrated players. Common random inputs can reduce variance, but do not force identical responses after histories diverge. BEN remains an external reference candidate; measure the integrated version and its conventions. [5]

Report paired IMP differences under the declared reference protocol, tricks, making probability and role-specific results, together with decision time, total compute and memory. Compare fast policy, deployed planner and teacher separately. Equal search-node counts alone do not establish equal practical cost. Keep the strength-versus-cost frontier as well as a headline rating.

Sweep decision budgets T, 2T and 4T. Report the marginal gain in tricks per board for each time doubling, with paired uncertainty, alongside the gain between engines at equal time. Include belief inference, sampling, search and reuse overhead in elapsed time. A faster solver matters when it shifts this whole-player curve; a higher trick score at a larger budget alone does not establish a better method.

12 Promotion and reliable measurement

A stronger league is more than a higher champion rating

Maintain a payoff matrix with uncertainty for informative matchups. A generalist must improve against the reference mixture without unacceptable regressions on protected slices. A specialist can be valuable because it exposes a champion weakness, improves coverage or supplies a distinctive partner. Beating current exploiters is not proof of low global exploitability.

Schedule matches near consequential decisions and uncertain interactions rather than continuously playing every pair. Freeze the evaluation panel within a season and retain historical anchors across seasons. Introduce a new panel by evaluating both old and new champions against it; otherwise a changed opponent mix can masquerade as progress.

Prevent statistical and benchmark overfitting

Measure paired variance before selecting sample counts. Predeclare fixed-sample tests or a justified sequential procedure, a practically relevant gain and regression tolerances. Account for repeated candidate selection and repeated looks; ordinary confidence intervals inspected after every batch are insufficient. Screening must preserve a path for slower-learning or uncertain high-potential ideas.

Split and resample by complete deal family when North-South or another fixed core is shared. Hold out opponent families and relevant style/convention combinations. Evaluate selected strategies on fresh world samples to reduce search-selection optimism. Separate inspectable development data, fresh promotion data and an audit stream unavailable to candidate authors.

Attribute each source of progress

Record three outcomes: strength at fixed decision time, strength reached at fixed training compute, and the learning curve as compute grows. For a training-method claim, compare matched starting points and multiple seeds. For two code improvements, compare baseline, A, B and A plus B before assuming benefits combine. Separate a good trained artifact from evidence of a reliably better training procedure.

Let failures create the next experiments

Preserve readable failure histories and component diagnostics. Better belief calibration with unchanged strength suggests a different next experiment from excessive sampling overhead or gains confined to familiar opponents. Development failures can become regression fixtures; promotion or audit cases enter training only through explicit retirement with a fresh replacement set.

13 Current work and path to a strong bot

The development path keeps a runnable player and repeatable arena at every stage. Neural learning and algorithmic research proceed in parallel behind stable interfaces. Evidence gates determine whether a component enters routine play, remains an expensive teacher or stays a research branch.

Status on 8 November 2026

Workstream	Current position	What is still needed
Harness and baseline	A working end-to-end harness with simple players, PIMC-DDS, replay and a BEN adapter has been reported.	Freeze reproducible versions; extend it into the registry and independent arena.
Consistent strategies	PIMC to alpha-mu-inspired implementation and tiny-ending checks are in progress.	Pinned regressions, tail-approximation accounting and broad equal-cost strength comparisons.
Related-deal reuse	Fixed-hand reuse studies and input validation for a related-deal batch solver are active work.	Sound applicability checks and net end-to-end gains across families and batch sizes.
Learning and league	The architecture and experimental programme are specified here.	Implement first belief training, frozen league epochs and controlled promotion.

This status distinguishes reported prototypes from promoted capability. Early alpha-mu comparisons do not yet establish a reliable general strength gain. Reuse or verification success does not establish net acceleration after overhead. Performance claims belong with a pinned code revision, corpus, hardware record and reproducible report.

First integration milestone

Freeze the current harness and PIMC player as anchors. Add immutable bot registration, leased jobs, paired comparisons and a small reference panel. Then submit one belief-only candidate and one search or solver candidate through exactly the same gate. The initial league can contain roughly 8 to 16 active bot bundles; archival retention is a separate decision.

The first useful result is a complete loop: a diagnosed failure produces an experiment, an agent or learner publishes a frozen candidate, the arena measures it on fresh families, and the league either admits it in a defined role or preserves the failure evidence. This infrastructure should support the present prototypes before larger training runs begin.

13 Capability milestones and dependencies

Milestone	Capability added	Gate before scaling
A Reproducible league	Working players, registry, independent evaluator, historical anchors and job leases.	Repeatable results, legal information use and restartable jobs.
B Better inference	Constrained belief net trained on versioned histories; simple player-type mixture.	Calibration and play gains on unseen families at equal elapsed time.
C Consistent search	Alpha-mu-inspired planner, exact tiny references and executable fallbacks.	Reduced fusion on known cases and a useful quality-cost curve.
D Efficient world batches	Validated fixed-hand batches; safe cache and hint reuse; checked generalizations.	Fresh-solve agreement and net gains including small-batch overhead.
E Fast learned play	Policy/value distillation, selective teacher calls and adaptive search allocation.	More strength per training hour and stable historical performance.
F Strong defense	Separate defender beliefs, shared agreements and diverse training partners.	Useful paired defense and generalization in cross-play.
G Broader capability	Richer deception, improved planning abstractions and possibly bidding.	A measured capability gap and evidence that the added complexity helps.

B, C and D can run concurrently after A provides trustworthy comparisons. C supplies better strategy teachers; D makes those teachers and deployed searches cheaper; B improves which worlds matter. E distills the resulting computation into routine play. Defensive information boundaries are present from the start, while full defensive capability develops through F.

How the target architecture emerges

Early versions can use a hand-coded policy, simple beliefs and selective alpha-mu search. As the belief net and distilled actor improve, search concentrates on close or consequential decisions. Related-deal reuse lowers its cost. A learned value function can then shorten search, and a controller can choose the appropriate budget. Preserve the high-budget teacher for cases the fast player still mishandles.

Keep alternatives alive when an experiment disappoints. If generalized proofs lose on overhead, retain exact caching and useful ordering hints. If full-hand strategy search is too costly, use it for selected positions or teaching. If type inference is weak, retain a broader mixture. Grow model capacity, data volume or hardware only after identifying a specific limiting resource.

14 Current hardware plan

The present plan is to begin on the MacBook Pro and move the main workload to one Mac Studio with substantially more memory. The hardware supports experimentation with bridge models and local engineering agents. It does not determine the search algorithm or establish a throughput target.

Tier	Current planning configuration	Intended role
Starting machine	14-inch MacBook Pro with 128 GB unified memory. In Apple's current M5 range, this corresponds to M5 Max, with 18 CPU and 40 GPU cores. [9]	Development, exact small cases, solver and inference benchmarks, and early learning runs.
Main machine	One M5 Ultra Mac Studio with 512 GB unified memory. Apple lists the corresponding 36-core CPU, 80-core GPU configuration and 1.2 TB/s memory bandwidth. [10]	Larger self-play and teacher workloads, model training, caches and selected local language-model agents.
Optional expansion	Additional compute only after measuring the limiting resource. Options include another Studio or a CUDA system.	More independent evaluation/self-play capacity, better training support, or larger local-model experiments.

These are the current configuration assumptions, not a claim about delivered hardware or a delivery date. Record the installed configuration and software stack when benchmarking. Do not infer a twofold solver rate merely from a twofold CPU-core count, or convert advertised GPU arithmetic directly into model throughput.

Memory and contention

The proposed bridge networks are small compared with either machine's memory. A 50-million-parameter model has about 100 MB of weights at 16 bits, although training adds gradients, optimizer state, activations and buffers. Replay, sampled layouts, solver contexts and local language models may matter more than the bridge weights.

A large local coding model can dominate memory and bandwidth. Start with frontier assistance where it is effective, and benchmark local models on completed engineering tasks rather than token rate alone. There is no requirement that the largest resident model and every training job run concurrently. Nor should the laptop be assumed unable to train a small bridge network while a modest local model is loaded.

Measure DDS, inference, training and local-agent workloads separately and together. Choose scheduling windows or concurrency limits from those results. MLX is a natural candidate on Apple Silicon; another supported framework or a CUDA machine remains an option if tooling or measured throughput justifies it.

Storage and scaling

Use compact versioned datasets and caches with measured hit rates. Keep code, evaluation results and valuable training checkpoints backed up. Full DD tables for enormous numbers of deals need not be generated before a learner has a demonstrated use for them.

Independent games and evaluations are natural candidates for distribution across machines. Splitting one large language model across several computers is a different workload with different communication requirements. Neither requires a cluster purchase as part of the initial bridge plan.

15 Research league and autonomous experiments

Max sets the objectives, budgets and important regression tolerances. Research agents implement bounded ideas; learners improve weights; an independent arena evaluates frozen submissions. The working system should automate routine execution while keeping experiment definitions, information boundaries and promotion evidence inspectable. The league is development infrastructure, separate from the deployed bot.

Two improvement loops and a slower selection loop

Loop	What changes	How progress is judged
Experience and learning	Belief, policy, value and response models under a versioned training recipe.	Fresh-family performance and learning curves per compute.
Algorithmic research	Search, representations, batching, reuse, objectives and training methods.	Controlled ablations and equal-budget comparisons.
Selection and integration	League membership, matchup weights and promoted bot bundles.	Independent confirmation, regression checks and role-specific value.

Learners may update frequently and agents may submit candidates continuously. Actors remain frozen within games and data epochs. League composition and opponent weights change at deliberate boundaries after evidence accumulates. This permits rapid experimentation without constantly moving the training target or comparison baseline.

Immutable bot and experiment records

A bot bundle identifies source revision, model hashes, configuration, search budget, observation schema, agreements, dependencies, build environment and license notices. Record parent bundles, data and training recipe, seed, component compatibility and evaluator version. A named bot must be reproducible later; an evolving learner publishes new identifiers rather than silently changing an old one.

The experiment record holds the hypothesis, motivating failures, predicted mechanism, primary metric, regression slices, compute cap, stopping rule and result uncertainty. Store compact results and data manifests with code; retain large datasets and checkpoints in appropriate artifact storage. Preserve provenance and access to the exact bytes needed to reproduce reported results.

Architecture can evolve without losing attribution

Give separate credit to the upstream DDS authors, alpha-mu researchers and other reused work. DDS is Apache 2.0 at the cited source; verify the license at each imported revision. The alpha-mu papers are algorithmic references, not a grant to copy an unidentified implementation. Record original contributions, borrowed code and AI-assisted engineering distinctly. [2,3,4]

15 Population retention and scheduling

Population	Purpose	Update policy
Archive	Reproduction, rollback, teachers and future research starting points.	Retain admitted bots, reported experiments and distinctive milestones; deduplicate artifacts.
Training league	Useful opponents and partners across strengths and styles.	Freeze an epoch; cap weight changes and retain historical representation.
Evaluation panel	Stable evidence of progress and regressions.	Freeze within a season; preserve anchors and bridge panel changes with common tests.

Why old and weaker bots remain useful

Keep historical champions, strategically distinct policies, exploiters that expose current weaknesses and partners with different conventions. Weaker checkpoints supply realistic error patterns and curriculum diversity. A bot can lose on average yet be the only one that detects a newly introduced weakness. Retention should consider payoff profiles and behavioral diversity alongside average score.

Archive membership does not imply frequent training use. Limit active inference and evaluation costs with a representative mixture, minimum representation for protected categories and occasional historical regression rounds. Candidate admission as a specialist is distinct from replacing the generalist champion. Retire active members only after their instructional or diagnostic role is covered; retain their reproducible records.

Matchmaking and gradual change

Initially sample from recent generalists, historical anchors, specialists and diverse partners with explicit weights. Freeze those weights for an epoch, then adjust them modestly using learning value, weaknesses and uncertainty. Give new bots limited initial weight. Avoid a rule that immediately copies the strongest checkpoint into every population slot.

Track a sparse payoff matrix and schedule the matchups that can change a consequential decision. PSRO offers a principled direction for training responses to policy mixtures; a simple mixture is sufficient initially. AlphaStar motivates keeping generalists and exploiters with different roles. Neither precedent establishes convergence or monotonic progress for this bridge system. [11,12]

Preserve diversity in research as well as opponents

Maintain multiple promising lineages when they differ in speed, strength, information use or learning potential. Cross compatible components only through tested interfaces, then evaluate the combination. A fresh model trained for a new search representation is not necessarily interchangeable with an older checkpoint merely because tensor dimensions match.

15 Work queue and experimental priorities

The lifecycle of an idea

Ideas enter with evidence, a parent bot, a falsifiable prediction and a budget. The scheduler assigns a lease to a research agent working in an isolated checkout. The agent submits code, tests, attribution and a runnable candidate. The arena screens and confirms it, then records promotion, specialist retention, further incubation or archival rejection. Leases expire and jobs checkpoint so interrupted work can resume without duplicate mutations.

Use explicit states such as proposed, leased, running, submitted, evaluated and resolved. Parent and dependency IDs make integration deliberate. Passing one experiment does not automatically merge a patch into every bot; the registry preserves the candidate while combination tests determine where it belongs.

Prioritize evidence and potential value

Allocate work using expected verified improvement relative to implementation, training and evaluation cost, together with information value and strategic coverage. Reserve an explicit exploration budget for uncertain ideas and separate incubation budgets for changes that need training before they can improve play. Do not let inexpensive parameter sweeps monopolize the queue.

Use test results to propose the next intervention. Calibration gains without playing gains suggest search or decision problems; excessive verification cost suggests a reuse interface or abstraction problem; gains limited to familiar policies suggest overfitting. Record failed hypotheses and conditions under which they were tested so later agents can distinguish a dead end from an idea worth revisiting.

Interactions between the two improvement loops

Measure code-only, weight-only and combined candidates where feasible. A faster solver may improve current search and produce more teacher labels; stronger beliefs may make those labels more relevant; better policies may reduce the cost of both play and training. Evaluate these compound effects through learning curves, not only an immediate tournament score.

AlphaEvolve supplies a useful model of coding agents, evolutionary program archives and staged evaluators. Our extension adds noisy competitive outcomes, partner compatibility and a deliberately stable historical population. Algorithmic novelty earns further experiments, while the independent arena determines the role of the resulting bot. [14]

Decisions that remain empirical

Open questions include the benefit of consistent search over strong PIMC, the breadth and net cost of reusable proofs, the hidden variables needed for good decisions, and the reliability of opponent inference. The model sizes and experience needed for expert strength are also unknown. The system should expose these uncertainties through replayable examples, controlled comparisons and documented learning curves.

16 References and evidence status

The sources support algorithmic precedents and available software. Model sizes, population sizes, data volumes and throughput scenarios remain design hypotheses or arithmetic illustrations. The project status in Section 13 distinguishes reported work in progress from promoted results. Newly added research references were reviewed for this revision on 8 November 2026.

[1] Matthew L Ginsberg. GIB Imperfect Information in a Computationally Challenging Game. Journal of Artificial Intelligence Research, 2001. Partition search and achievable strategies under imperfect information.
https://www.cs.cmu.edu/afs/cs/project/jair/pub/volume14/ginsberg01a.pdf

[2] Tristan Cazenave and Veronique Ventos. The alpha mu Search Algorithm for the Game of Bridge. A relevant strategy-search baseline with perfect-information opponents.
https://www.lamsade.dauphine.fr/~cazenave/papers/AlphaMu.pdf

[3] Tristan Cazenave, Swann Legras and Veronique Ventos. Optimizing alpha mu. 2021. Pareto-front pruning and redundant-world optimizations.
https://www.lamsade.dauphine.fr/~cazenave/papers/OptimizingAlphaMu.pdf

[4] DDS project. Double Dummy Solver. Pin a repository revision and toolchain for each benchmark.
https://github.com/dds-bridge/dds

[5] Lorand Dali and contributors. BEN bridge engine. Candidate external bidder and player; evaluate the integrated version and its conventions.
https://github.com/lorserker/ben

[6] Daochen Zha and coauthors. DouZero Mastering DouDizhu with Self Play Deep Reinforcement Learning. ICML, 2021. An adjacent card-game learning precedent.
https://proceedings.mlr.press/v139/zha21a.html

[7] Guan Yang and coauthors. PerfectDou Dominating DouDizhu with Perfect Information Distillation. NeurIPS, 2022. Privileged training information with information-limited execution.
https://arxiv.org/abs/2203.16406

[8] Sam Ganzfried and Qingyun Sun. Bayesian Opponent Exploitation in Imperfect Information Games. A precedent for updating opponent models from observed play.
https://arxiv.org/abs/1603.03491

[9] Apple. MacBook Pro technical specifications. Current 128 GB M5 Max configuration.
https://www.apple.com/macbook-pro/specs/

[10] Apple. Mac Studio technical specifications. Current 512 GB M5 Ultra configuration.
https://www.apple.com/mac-studio/specs/

16 League and learning references

[11] Oriol Vinyals and coauthors. Grandmaster level in StarCraft II using multi-agent reinforcement learning. Nature, 2019. Generalists, exploiters and league training.

https://deepmind.google/blog/alphastar-grandmaster-level-in-starcraft-ii-using-multi-agent-reinforcement-learning/

[12] Marc Lanctot and coauthors. A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning. NeurIPS, 2017. Policy-space response oracles and mixtures of policies.

https://mlanctot.info/files/papers/nips17-psro.pdf

[13] Noam Brown, Anton Bakhtin, Adam Lerer and Qucheng Gong. Combining Deep Reinforcement Learning and Search for Imperfect-Information Games. NeurIPS, 2020. ReBeL and belief-based search; two-player zero-sum setting.

https://arxiv.org/abs/2007.13544

[14] Alexander Novikov and coauthors. AlphaEvolve A coding agent for scientific and algorithmic discovery. 2025. Evaluated program evolution, diversity and evaluation cascades.

https://arxiv.org/abs/2506.13131

[15] Z. Qiu, S. Wang, D. You and M. C. Zhou. Bridge Bidding via Deep Reinforcement Learning and Belief Monte Carlo Search. IEEE/CAA Journal of Automatica Sinica, 2024. Belief supervision from self-play; bidding rather than card play.

https://doi.org/10.1109/JAS.2024.124488

[16] Hengyuan Hu, Adam Lerer, Noam Brown and Jakob Foerster. Learned Belief Search Efficiently Improving Policies in Partially Observable Settings. 2021. Learned autoregressive beliefs and efficient rollouts in Hanabi.

https://arxiv.org/abs/2106.09086

[17] DJ Strouse and coauthors. Collaborating with Humans without Human Data. NeurIPS, 2021. Fictitious Co-Play with diverse partners and historical checkpoints.

https://arxiv.org/abs/2110.08176

[18] Hengyuan Hu and coauthors. Off-Belief Learning. ICML, 2021. Fragile self-play conventions and coordination with unfamiliar agents.

https://proceedings.mlr.press/v139/hu21c.html